

Module 3 Key management and Distribution

Symmetric Key Distribution Using Symmetric Encryption

For **symmetric** encryption to work, the two parties to an exchange must share the same key, and that key must be protected from access by others. Therefore, the term that refers to the means of delivering a key to two parties who wish to exchange data, without allowing others to see the key. For two parties A and B, key distribution can be achieved in a number of ways, as follows:

1. A can select a key and physically deliver it to B.
2. A third party can select the key and physically deliver it to A and B.
3. If A and B have previously and recently used a key, one party can transmit the new key to the other, encrypted using the old key.
4. If A and B each has an encrypted connection to a third party C, C can deliver a key on the encrypted links to A and B.

Physical delivery (1 & 2) is simplest - but only applicable when there is personal contact between recipient and key issuer. This is fine for link encryption where devices & keys occur in pairs, but does not scale as number of parties who wish to communicate grows. 3 is mostly based on 1 or 2 occurring first.

A third party, whom all parties trust, can be used as a **trusted intermediary** to mediate the establishment of secure communications between them (4). Must trust intermediary not to abuse the knowledge of all session keys. As number of parties grow, some variant of 4 is only practical solution to the huge growth in number of keys potentially needed.

Key distribution centre:

- The use of a **key distribution center** is based on the use of a hierarchy of keys. At a minimum, two levels of keys are used.
- Communication between end systems is encrypted using a temporary key, often referred to as a **Session key**.
- Typically, the session key is used for the duration of a logical connection and then discarded
- **Master key** is shared by the key distribution center and an end system or user and used to encrypt the session key.

Key Distribution Scenario:

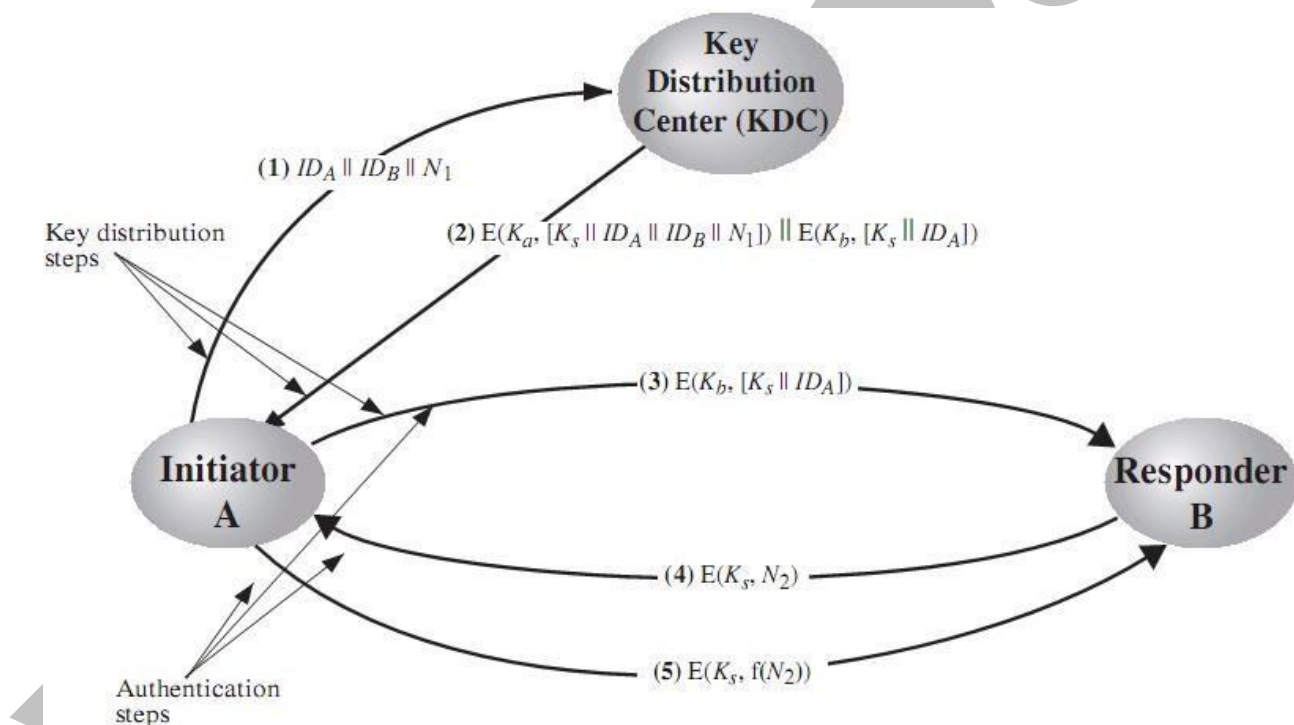


Figure 14.3 Key Distribution Scenario

Let us assume that user A wishes to establish a logical connection with B and requires a one-time session key to protect the data transmitted over the connection. A has a master key, K_a , known only to itself and the KDC; similarly, B shares the master key K_b with the KDC. The following steps occur:

- 1 A issues a request to the KDC for a session key to protect a logical connection to B. The message includes the identity of A and B and a unique identifier, N_1 , for this transaction, which we refer to as a **nonce**. The nonce may be a timestamp, a counter, or a random number; the minimum requirement is **that it differs with each request**. Also, to prevent masquerade, it should be difficult for an opponent to guess the nonce. Thus, a random number is a good choice for a nonce.
2. The KDC responds with a message encrypted using K_a . Thus, A is the only one who can successfully read the message, and A knows that it originated at the KDC. The message includes two items intended for A:
 - The **one-time session key, K_s** , to be used for the session
 - The **original request message**, including the nonce, to enable A to match this response with the appropriate request

Thus, A can verify that its original request was not altered before reception by the KDC and, because of the nonce, that this is not a replay of some previous request. In addition, the message includes two items intended for B:

- The one-time session key, K_s to be used for the session
- An identifier of A (e.g., its network address), IDA

These last two items are encrypted with K_b (the master key that the KDC shares with B). They are to be sent to B to establish the connection and prove A's identity.

3. A stores the session key for use in the upcoming session and forwards to B the information that originated at the KDC for B, namely, $E(K_b, [K_s \parallel IDA])$. Because this information is encrypted with K_b , it is protected from eavesdropping. B now knows the session key (K_s), knows that the other party is A (from IDA), and knows that the information originated at the KDC (because it is encrypted using K_b). At this point, a session key has been securely delivered to A and B, and they may begin

their protected exchange. However, two additional steps are desirable:

4. Using the newly minted session key for encryption, B sends a nonce, N_2 , to A.
5. Also using K_s , A responds with $f(N_2)$, where f is a function that performs some transformation on N_2 (e.g., adding one).

These steps assure B that the original message it received (step 3) was not a replay.

Note that the actual key distribution involves only steps 1 through 3 but that steps 4 and 5, as well as 3, perform an authentication function.

Major Issues with KDC:

1. Hierarchical Key Control

- ☐ It is not **necessary to limit** the key distribution function to a single KDC. Indeed, for very large networks, it may not be practical to do so. As an alternative, a hierarchy of KDCs can be established.
- ☐ For example, there can be local KDCs, each responsible for a small domain of the overall internetwork, such as a single LAN or a single building.
- ☐ If two entities in different domains desire a shared key, then the corresponding local KDCs can communicate through a global KDC.
- ☐ The hierarchical concept can be extended to three or even more layers, depending on the size of the user population and the geographic scope of the internetwork.
- ☐ A hierarchical scheme **minimizes the effort involved in master key distribution**, because most master keys are those shared by a local KDC with its local entities.

2. Session Key Lifetime

- ☐ The distribution of session keys delays the start of any exchange and places a burden on network capacity. A security manager must try to balance these competing considerations in determining the lifetime of a particular session key.

- ❑ For **connection-oriented protocols**, one obvious choice is to use the same session key for the length of time that the connection is open, using a new session key for each new session.
- ❑ If a logical connection has a very long lifetime, then it would be prudent to change the session key periodically, perhaps every time the PDU (protocol data unit) sequence number cycles.
- ❑ For a **connectionless protocol**, such as a transaction-oriented protocol, there is no explicit connection initiation or termination.
- ❑ Thus, it is not obvious how often one needs to change the session key. The most secure approach is to use a **new session key for each exchange**.
- ❑ A better strategy is to use a given **session key for a certain fixed period only or for a certain number of transactions**.

3. A Transparent Key Control Scheme

- ❑ The approach suggested in Figure 14.3 is useful for providing end-to-end encryption at a network or transport level in a way that is transparent to the end users.
- ❑ The approach assumes that communication makes use of a connection-oriented end-to-end protocol, such as TCP.
- ❑ The noteworthy element of this approach is a session security module (SSM), which may consist of functionality at one protocol layer, that performs end-to-end encryption and obtains session keys on behalf of its host or terminal.

The steps involved in establishing a connection are shown in Figure 14.4.

1. When one host wishes to set up a connection to another host, it transmits a connection-request packet.
2. The SSM saves that packet and applies to the KDC for permission to establish the connection.
3. The communication between the SSM and the KDC is encrypted using a master key shared only by this SSM and the KDC. If the KDC approves the

connection request, it generates the session key and delivers it to the two appropriate SSMs, using a unique permanent key for each SSM.

4. The requesting SSM can now release the connection request packet, and a connection is set up between the two end systems.
5. All user data exchanged between the two end systems are encrypted by their respective SSMs using the onetime session key.

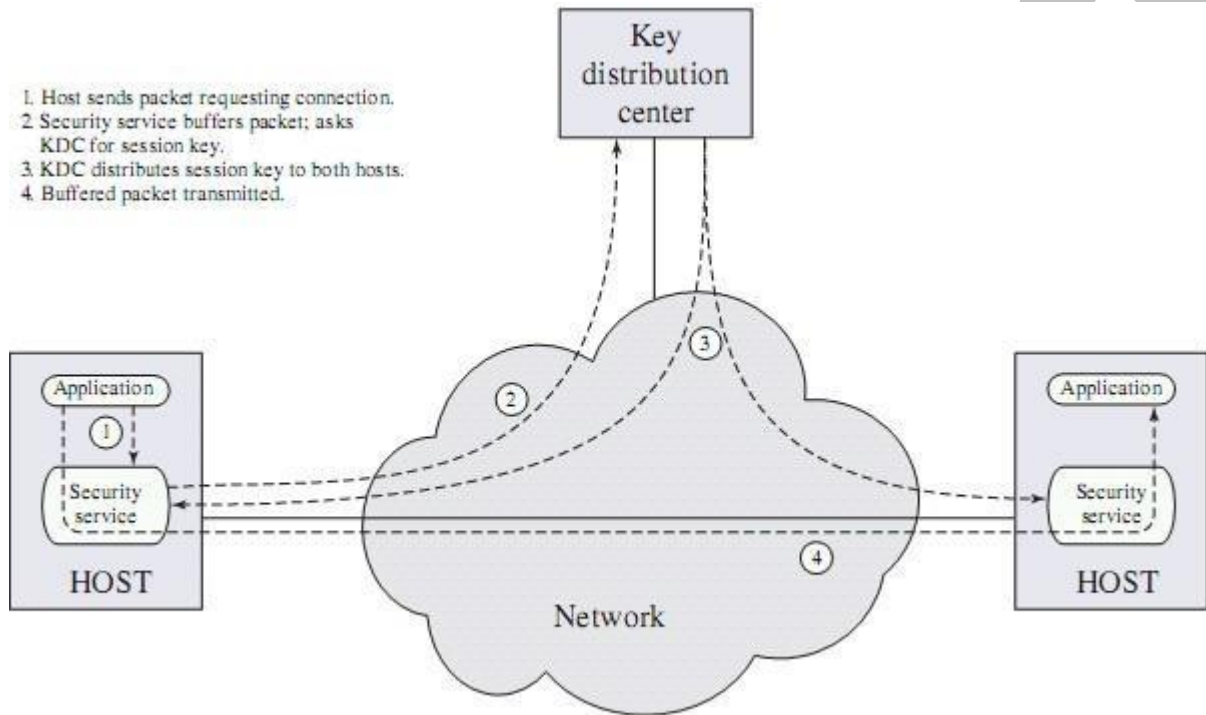


Figure 14.4 Automatic Key Distribution for Connection-Oriented Protocol

- The automated key distribution approach provides the flexibility and dynamic characteristics needed to allow a number of terminal users to access a number of hosts and for the hosts to exchange data with each other.

4. Decentralized Key Control

- The use of a key distribution center imposes the requirement that the KDC be trusted and be protected from subversion. This requirement can be avoided if key distribution is fully decentralized.

- Although full decentralization is not practical for larger networks using symmetric encryption only, it may be useful within a local context.
- A decentralized approach requires that each end system be able to communicate in a secure manner with all potential partner end systems for purposes of session key distribution.
- Thus, there may need to be as many as $n(n-1)/2$ master keys for a configuration with n end systems.
- A session key may be established with the following sequence of steps (Figure 14.5).
 1. A issues a request to B for a session key and includes a nonce, .
 2. B responds with a message that is encrypted using the shared master key. The response includes the session key selected by B, an identifier of B, the value $f(N_1)$, and another nonce N_2 .
 3. Using the new session key, A returns $f(N_2)$ to B.

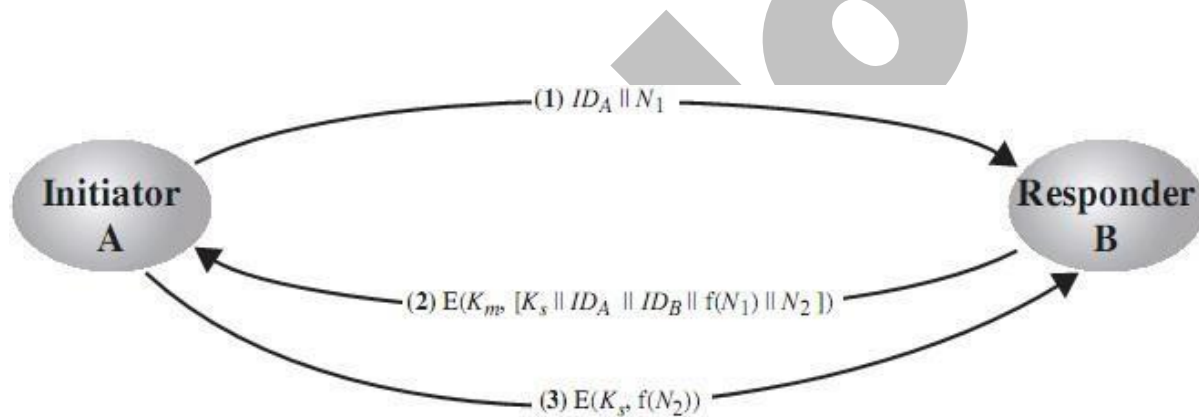


Figure 14.5 Decentralized Key Distribution

5. Controlling Key Usage

The concept of a key hierarchy and the use of automated key distribution techniques greatly reduce the number of keys that must be manually managed and distributed. It also may be desirable to impose some control on the way in which automatically distributed keys are used. For example, in addition to separating master keys from session keys, we may wish to define different types of session keys on the basis of use, such as

- Data-encrypting key, for general communication across a network
- PIN-encrypting key, for personal identification numbers (PINs) used in electronic funds transfer and point-of-sale applications
- File-encrypting key, for encrypting files stored in publicly accessible locations

To illustrate the value of separating keys by type, consider the risk that a master key is imported as a data-encrypting key into a device. Normally, the master key is physically secured within the cryptographic hardware of the key distribution center and of the end systems. Session keys encrypted with this master key are available to application programs, as are the data encrypted with such session keys.

However, if a master key is treated as a session key, it may be possible for an unauthorized application to obtain plaintext of session keys encrypted with that master key.

The proposed technique is for use with DES and makes use of the extra 8 bits in each 64-bit DES key. That is, the eight non-key bits ordinarily reserved for parity checking form the key tag. The bits have the following interpretation:

- One bit indicates whether the key is a session key or a master key.
- One bit indicates whether the key can be used for encryption.
- One bit indicates whether the key can be used for decryption.
- The remaining bits are spares for future use.

Because the tag is embedded in the key, it is encrypted along with the key when that key is distributed, thus providing protection. The drawbacks of this scheme are

1. The tag length is limited to 8 bits, limiting its flexibility and functionality.
2. Because the tag is not transmitted in clear form, it can be used only at the point of decryption, limiting the ways in which key use can be controlled.

A more flexible scheme, referred to as the control vector, is described here. In this scheme, each session key has an associated control vector consisting of a number of fields

that specify the uses and restrictions for that session key. The length of the control vector may vary.

The control vector is cryptographically coupled with the key at the time of key generation at the KDC.

As a first step, the control vector is passed through a hash function that produces a value whose length is equal to the encryption key length. In essence, a hash function maps values from a larger range into a smaller range with a reasonably uniform spread. Thus, for example, if numbers in the range 1 to 100 are hashed into numbers in the range 1 to 10, approximately 10% of the source values should map into each of the target values. The hash value is then XORed with the master key to produce an output that is used as the key input for encrypting the session key. Thus,

$$\text{Hash value} = H = h(\text{CV}) \quad \text{Key input} = K_m \oplus H$$

$$\text{Ciphertext} = E([K_m \oplus H], K_s)$$

where K_m is the master key and K_s is the session key. The session key is recovered in plaintext by the reverse operation:

$$D([K_m \oplus H], E([K_m \oplus H], K_s))$$

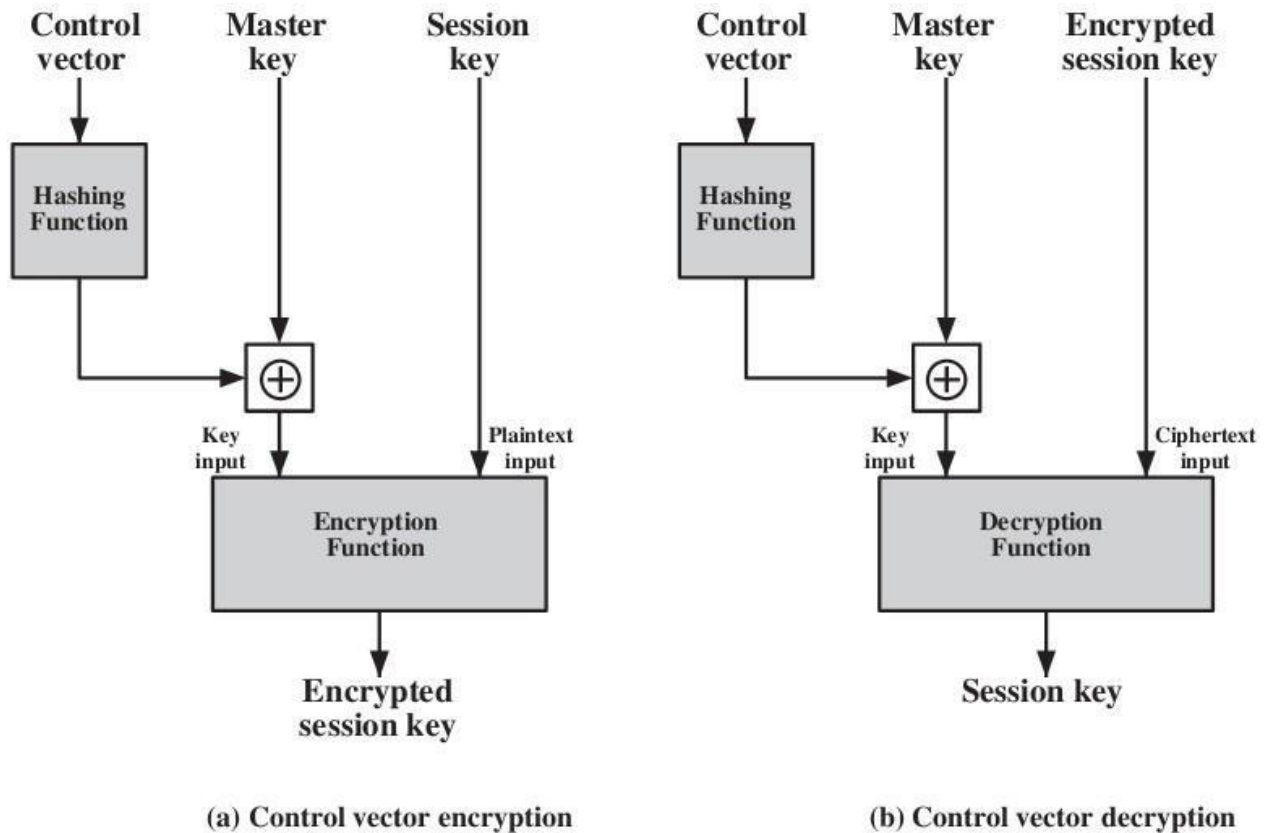


Figure 14.6 Control Vector Encryption and Decryption

When a session key is delivered to a user from the KDC, it is accompanied by the control vector in clear form. The session key can be recovered only by using both the master key that the user shares with the KDC and the control vector. Thus, the linkage between the session key and its control vector is maintained.

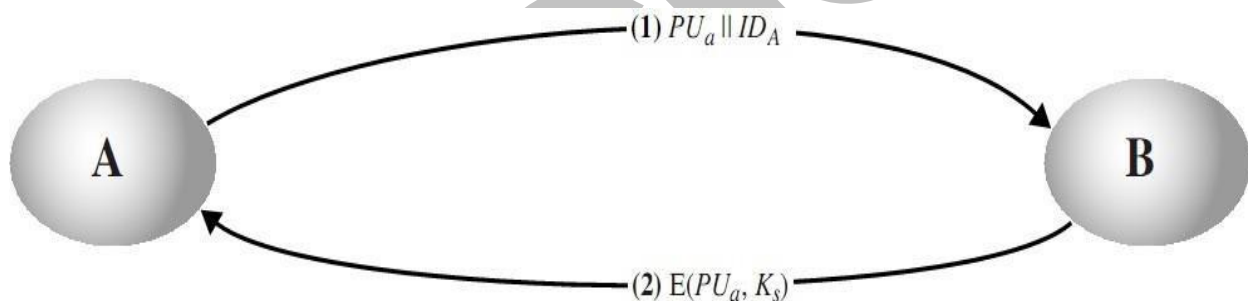
Use of the control vector has two **advantages over use of an 8-bit tag**. First, there is no restriction on length of the control vector, which enables arbitrarily complex controls to be imposed on key use. Second, the control vector is available in clear form at all stages of operation. Thus, control of key use can be exercised in multiple locations.

SYMMETRIC KEY DISTRIBUTION USING ASYMMETRIC ENCRYPTION

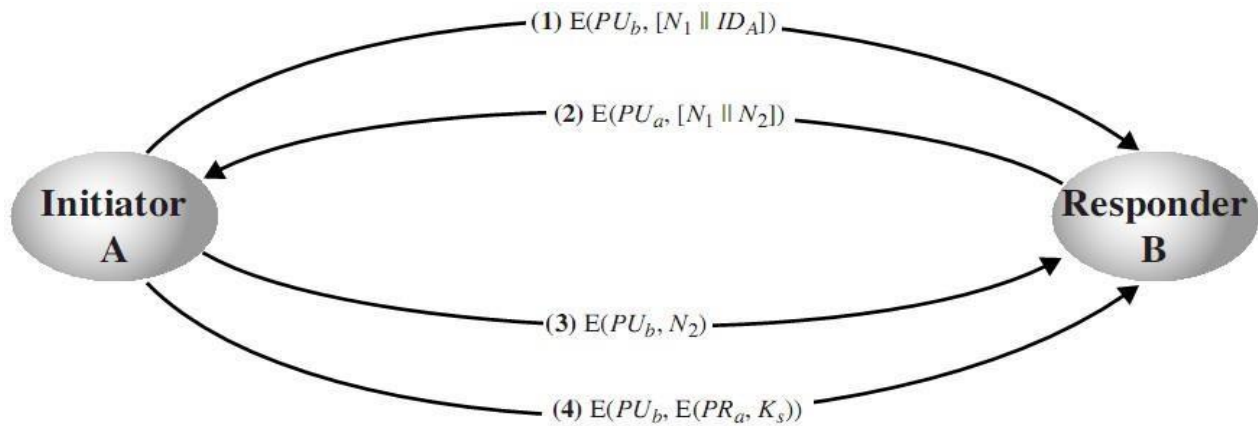
- Once public keys have been distributed or have become accessible, secure communication that thwarts eavesdropping, tampering, or both, is possible.
- Public-key encryption provides for the distribution of secret keys to be used for conventional encryption.

Simple Secret Key Distribution

- A generates a public/private key pair $\{PU_a, PR_a\}$ and transmits a message to B consisting of PU_a and an identifier of A, ID_A
- B generates a secret key, K_s , and transmits it to A, encrypted with A's public key.
- A computes $D(PR_a, E(PU_a, K_s))$ to recover the secret key. Because only A can decrypt the message, only A and B will know the identity of K_s .
- A discards PU_a and PR_a and B discards PU_a .



Here third party can intercept messages and then either relay the intercepted message or substitute another message. Such an attack is known as a **man-in-the-middle attack**.

Secret Key Distribution with Confidentiality and Authentication:

- ☐ A uses B's public key to encrypt a message to B containing an identifier of A (ID_A) and a nonce (N_1), which is used to identify this transaction uniquely
- ☐ B sends a message to A encrypted with PU_a and containing A's nonce (N_1) as well as a new nonce generated by B (N_2). Because only B could have decrypted message (1), the presence of N_1 in message (2) assures A that the correspondent is B.
- ☐ A returns N_2 encrypted using B's public key, to assure B that its correspondent is A.
- ☐ A selects a secret key K_s and sends $M = E(PU_b, E(PR_a, K_s))$ to B. Encryption of this message with B's public key ensures that only B can read it; encryption with A's private key ensures that only A could have sent it.
- ☐ B computes $D(PU_a, D(PR_b, M))$ to recover the secret key.

A Hybrid Scheme:

Yet another way to use public-key encryption to distribute secret keys is a hybrid approach.

- This scheme retains the use of a key distribution center (KDC) that shares a secret master key with each user and distributes secret session keys encrypted with the master key.
- A public key scheme is used to distribute the master keys.
- The addition of a public-key layer provides a secure, efficient means of distributing master keys.

Distribution of Public Keys:

Several techniques have been proposed for the distribution of public keys, which can mostly be grouped into the categories shown.

- ☐ Public announcement
- ☐ Publicly available directory
- ☐ Public-key authority
- ☐ Public-key certificates

Public Announcement of Public Keys

The point of public-key encryption is that the public key is public, hence any participant can send his or her public key to any other participant, or broadcast the key to the community at large. eg. append PGP keys to email messages or post to news groups or email list

Its major weakness is forgery, anyone could pretend to be user A and send a public key to

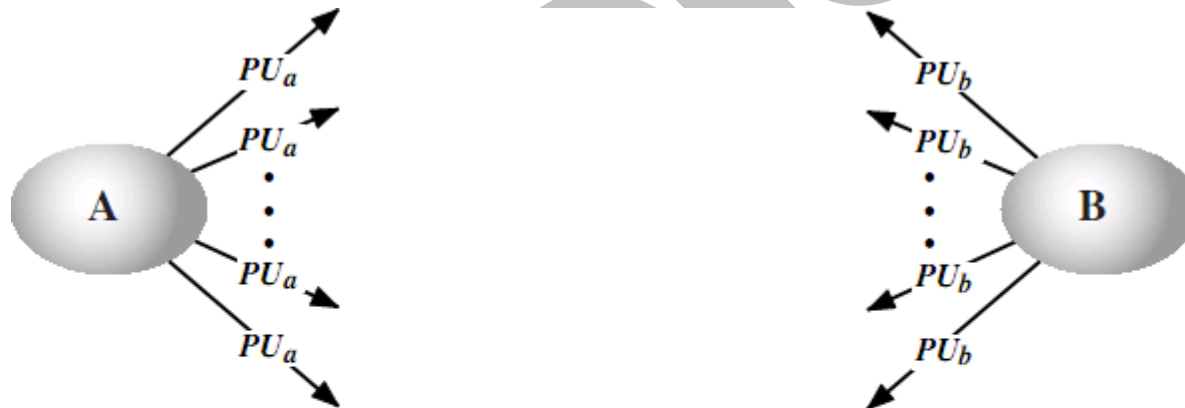
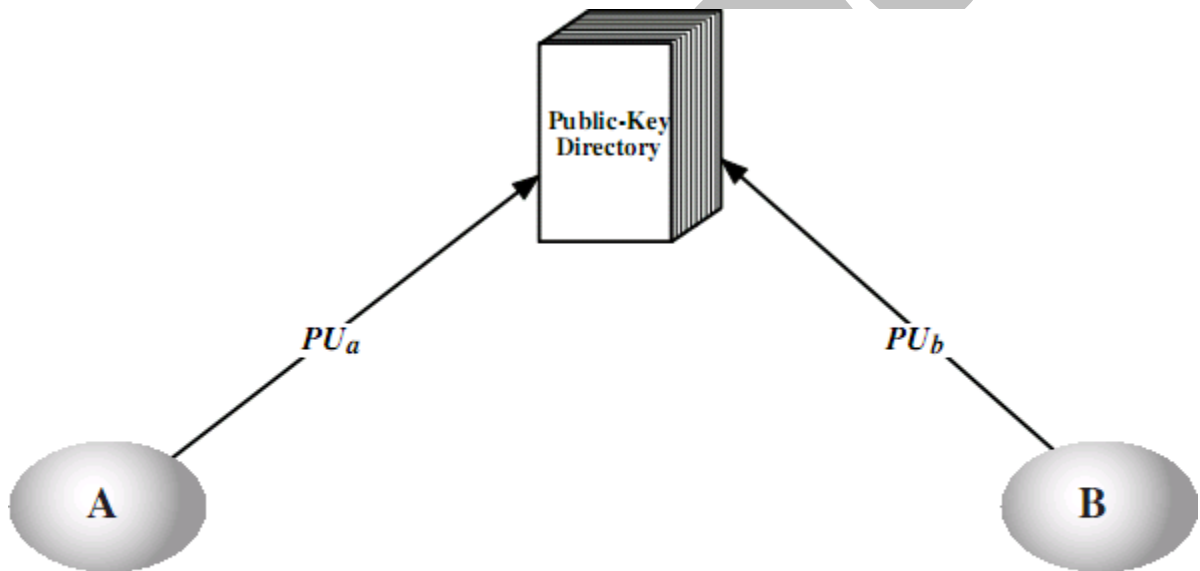


Figure 10.1 Uncontrolled Public Key Distribution

another participant or broadcast such a public key. Until the forgery is discovered they can masquerade as the claimed user.

Publicly Available Directory

- can obtain greater security by registering keys with a public directory
- directory must be trusted with properties:
 - The authority maintains a directory with a {name, public key} entry for each participant.
 - Each participant registers a public key with the directory authority.
 - A participant may replace the existing key with a new one at any time because the corresponding private key has been compromised in some way.
 - Participants could also access the directory electronically. For this purpose, secure, authenticated communication from the authority to the participant is mandatory.

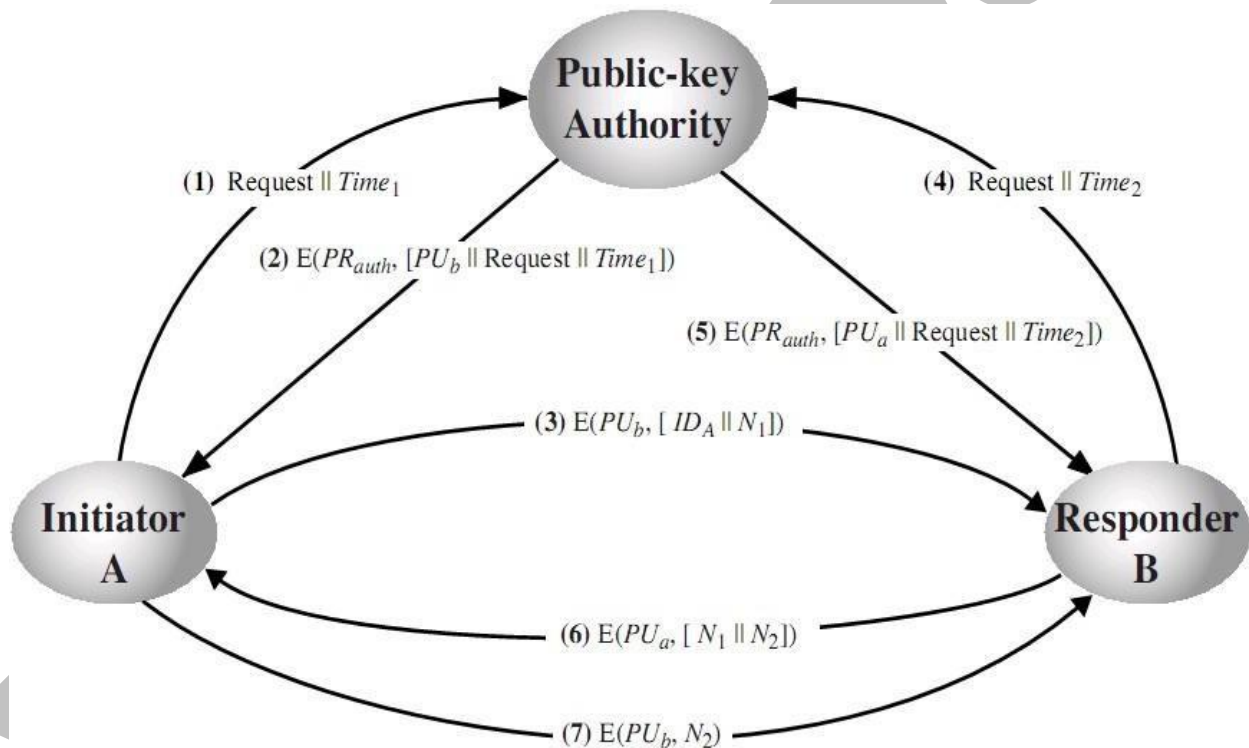
**Figure 10.2 Public Key Publication**

This scheme is clearly more secure than individual public announcements but still has vulnerabilities.

If an adversary succeeds in obtaining or computing the private key of the directory authority, the adversary could authoritatively pass out counterfeit public keys and subsequently impersonate any participant and eavesdrop on messages sent to any participant. Another way to achieve the same end is for the adversary to tamper with the records kept by the authority.

Public-Key Authority:

- Stronger security for public-key distribution can be achieved by providing tighter control over the distribution of public keys from the directory.
- It requires users to know the public key for the directory, and that they interact with directory in real-time to obtain any desired public key securely.
- Totally seven messages are required.



1. A sends a timestamped message to the public-key authority containing a request for the current public key of B.
2. The authority responds with a message that is encrypted using the authority's private key, PR_{auth} . Thus, A is able to decrypt the message using the authority's public key. Therefore, A is assured that the message originated with the authority. The message includes the following:
 - B's public key, PU_B which A can use to encrypt messages destined for B
 - The original request, to enable A to match this response with the corresponding earlier request and to verify that the original request was not altered before reception by the authority.
 - The original timestamp, so A can determine that this is not an old message from the authority containing a key other than B's current public key.
3. A stores B's public key and also uses it to encrypt a message to B containing an identifier of A (IDA) and a nonce ($N1$), which is used to identify this transaction uniquely.
4. B retrieves A's public key from the authority in the same manner as A retrieved B's public key.
5. At this point, public keys have been securely delivered to A and B, and they may begin their protected exchange. However, two additional steps are desirable:
6. B sends a message to A encrypted with PU_A and containing A's nonce ($N1$) as well as a new nonce generated by B ($N2$). Because only B could have decrypted message (3), the presence of $N1$ in message (6) assures A that the correspondent is B.
7. A returns $N2$, encrypted using B's public key, to assure B that its correspondent is A.

Public-Key Certificates

- ❑ A user must appeal to the authority for a public key for every other user that it wishes to contact and it is vulnerable to tampering too.
- ❑ Public key certificates can be used to exchange keys without contacting a public-key authority.
- ❑ A certificate binds an **identity** to **public key**, with all contents **signed** by a trusted Public-Key or Certificate Authority (CA).
- ❑ This can be verified by anyone who knows the public-key authorities public-key.

A participant can also convey its key information to another by transmitting its certificate.

Other participants can verify that the certificate was created by the authority. We can place the following requirements on this scheme:

1. Any participant can read a certificate to determine the name and public key of the certificate's owner.
2. Any participant can verify that the certificate originated from the certificate authority and is not counterfeit.
3. Only the certificate authority can create and update certificates.
4. Any participant can verify the currency of the certificate.

One scheme has become universally accepted for formatting public-key certificates: the X.509 standard.

X.509 certificates are used in most network security applications, including IP security, secure sockets layer (SSL), secure electronic transactions (SET), and S/MIME.

